

HVAC AUTO-ESTIMATION SYSTEM

NEXT-GENERATION OVERHAUL PLAN

VERSION 2.0 — CFD · 3D · UI · ENGINE · STACK

Repository: Ackerman-G4/HVAC-Auto-Est-Project | Branch: main-backup2 @ 06b83d5

Prepared: July 2026 | Supersedes: HVAC System Overhaul Plan v3.0 (June 2026)

Basis: full source-code audit of the live branch, post Waves 1–6

*Every claim in this document about the current codebase was verified by reading or executing the code on 2026-07-06.
Items that are design targets rather than verified facts are labeled as such.*

Table of Contents

1. Executive Summary	3
1.1 Where the platform actually stands (verified 2026-07-06)	3
1.2 What v2.0 changes relative to the v3.0 document	3
2. Guiding Engineering Decisions	4
D1 — OpenFOAM is the engineering-grade solver. Do not build a custom SST k- ω	4
D2 — The tier split is a product feature, not a compromise	5
D3 — No Rust/WASM until profiling demands it	5
D4 — WebGL2 now, WebGPU as progressive enhancement later	5
D5 — No new queue infrastructure	5
D6 — Languages	5
3. Target Architecture	6
4. CFD Engine — From Scaffold to Engineering-Grade	6
4.1 Phase C1 — Prove the solver locally (gate: one real room solves end-to-end)	6
4.2 Phase C2 — Cloud infrastructure	6
4.3 Phase C3 — Orchestration (the two NEW routes)	7
4.4 Phase C4 — Validation ladder (the actual engineering-grade gate)	7
4.5 Phase C5 — Real geometry (snappyHexMesh)	7
4.6 Explicit non-goals for the CFD engine in v2.0	8
5. 3D Visualization — React Three Fiber Overhaul	8
5.1 Migration strategy — strangler, not rewrite	8
5.2 Layer specifications	8
5.3 Performance budgets (hard limits, checked in review)	9
6. UI & Animation System	9
6.1 Motion principles	9
6.2 The CFD tile & simulation surfaces	9
6.3 Page-level polish pass (ordered by client visibility)	10
7. Remaining Engine Work (non-CFD)	10
8. Technology Stack After v2.0	11
9. Implementation Timeline — sized for one developer	11
10. Testing, Verification & Quality Gates	12
11. Risk Register	13
12. Professional Assessment	13

1. Executive Summary

This plan is the successor to the v3.0 Security-Hardened plan. Most of v3.0 has now shipped: the birthday login is deleted, the real equipment catalog is wired in, price overrides are admin-gated and audited, BOQ snapshots are hash-locked, the admin portal exists with five live tabs, per-user project ownership is enforced, lockout and password policy are live, and — as of Wave 6 — price overrides actually flow into quoted prices, login lockout is IP-aware, and the OpenFOAM export path produces structurally valid solver cases with a containerized Cloud Run solver written and unit-checked.

What remains is the part v3.0 sketched most ambitiously and least concretely: the CFD engine, the 3D view, and the interface that presents them. This document replaces v3.0's Section 8 (CFD & 3D) and Section 5 (UI) with a design that is grounded in the code that actually exists, corrects three of v3.0's technology choices that would have cost months for negative return, and defines the exact sequence to a system you can put in front of a paying client.

How to read this document

Sections 2 and 3 are decisions and rationale — read them once, then hold the rest of the plan accountable to them. Sections 4–7 are the build plans (CFD, 3D, UI, remaining engine work). Section 8 is the stack. Section 9 is the timeline sized for one developer. Section 10 is the validation gate that separates "it runs" from "engineering-grade."

1.1 Where the platform actually stands (verified 2026-07-06)

Area	Status	Evidence
Auth & security	SHIPPED	JWT+middleware RBAC, email+IP lockout, password policy, forgot-password, login events with indexed queries
Financial integrity	SHIPPED	Admin price API (capped, justified, dual audit), overrides now reach sizing & quotes, server-resolved prices for catalog SKUs, SHA-256 BOQ snapshots + verify endpoint
Admin portal	SHIPPED (read-only users)	/admin with Dashboard, Users, Projects, Audit Log, Price Controls
Load calculation	SHIPPED (CLTD/CLF)	Component-based CLTD/CLF drives TR; PH city design conditions auto-populate; area rule kept as floor
BOQ	PARTIAL	Equipment, piping, drain, electrical, ductwork wired; Diffusers, Controls, itemized fees still missing (Section 7)
In-browser CFD	SHIPPED	Hand-rolled SIMPLE + $k-\epsilon$ with Rhie–Chow, humidity transport, building-level multi-room solver fed by real floor/room geometry
OpenFOAM path	SCAFFOLDED	Exporter emits valid cases (boundary faces, topoSet/createPatch, buoyant thermophysics, gravity); container + wrapper written and logic-tested; never run against a live solver
3D view	SHIPPED (raw Three.js)	BuildingSimulationViewer3D + CFDOverlay3D render real project geometry, prop-driven, with particle overlays

1.2 What v2.0 changes relative to the v3.0 document

v3.0 said	v2.0 decision	Why (short form — full rationale in Section 2)
Custom Python SST k- ω solver with Numba JIT and hand-built octree AMR	OpenFOAM as the engineering-grade solver; keep the TS solver as the free Preview tier	SST k- ω , AMR-class meshing, and 20 years of validation come free in OpenFOAM. Hand-building them solo is months of work producing an unvalidatable solver.
Rust/WASM geometry engine for polygon extrusion	Keep the existing TypeScript extruder	room-extruder.ts already works and is not a measured bottleneck. Rewriting working code in a new language is risk with no payoff until profiling says otherwise.
WebGPU rendering	WebGL2 (Three.js/R3F) now; WebGPU as later progressive enhancement	WebGL2 runs everywhere your clients are, including office laptops and tablets. Volumetric raymarching works fine in WebGL2 at this mesh scale.
BullMQ + Upstash Redis job queue	Cloud Run Jobs + Firestore runJobs as the queue and state	The job system already exists in simulation-cases-store.ts. Adding Redis is a second infrastructure to pay for, secure, and babysit, to solve a problem Cloud Run Jobs already solves.
FastAPI WebSocket streaming of solver progress	Firestore-driven polling (already built) now; callback + snapshots for OpenFOAM	Snapshot/polling infra already exists end-to-end. WebSockets add a stateful server to host; revisit only if polling latency actually hurts UX.

2. Guiding Engineering Decisions

These five decisions shape everything else in the plan. Each one is stated with its reasoning so it can be challenged on the merits later — a decision you can't interrogate is a superstition, not engineering.

D1 — OpenFOAM is the engineering-grade solver. Do not build a custom SST k- ω .

v3.0 called for a from-scratch Python solver with SST k- ω turbulence, octree adaptive mesh refinement, and conjugate heat transfer, JIT-compiled with Numba. Each of those is individually a serious project; together they are a research codebase. Worse, a custom solver has no validation pedigree — when a client's mechanical engineer asks "what solver produced this," the answer "one I wrote myself" ends the conversation, while "OpenFOAM, the same platform used across the HVAC and automotive industries" starts one.

The decisive detail: the exporter built in Wave 6 already maps the turbulence model selection to OpenFOAM — including k-omega-sst → kOmegaSST. SST k- ω , the headline feature of v3.0's custom-solver plan, is a dropdown option on the OpenFOAM path once that path is validated. You get v3.0's accuracy goal with zero solver-development effort.

- Keep the existing TypeScript SIMPLE/k- ϵ in-browser solver as the free Preview tier. It runs instantly, costs nothing, and is the only path with humidity transport — a genuine advantage in Philippine psychrometrics.
- OpenFOAM (buoyantSimpleFoam, later kOmegaSST) is the paid Engineering tier: slower, costs compute, produces defensible numbers.

- Every simulation result is permanently labeled with its solver tier. Never let a Preview number appear where an Engineering number is expected.

D2 — The tier split is a product feature, not a compromise

Preview vs. Engineering maps directly onto the SaaS pricing tiers already planned in the commercial layer: free users iterate quickly with Preview; paying users unlock Engineering runs. The same geometry, cases, and 3D viewer serve both — only the solver behind the run changes. This is why the SimulationResult/RunJob contract must stay solver-agnostic, which the existing runSource field (internal | openfoam | simflow | manual-import) already supports.

D3 — No Rust/WASM until profiling demands it

The geometry pipeline (parseRoomPolygon → extrudeRoomFromPolygon → detectAdjacentRooms → buildBuildingGeometryFromFloors) is working TypeScript operating on tens of rooms with tens of vertices each. That is microseconds of work. A Rust/WASM rewrite adds a toolchain, a build step, a language, and an FFI boundary to a part of the system that has never appeared in a performance trace. Rule: no rewrite without a profile showing the extruder above 10% of frame or request time.

D4 — WebGL2 now, WebGPU as progressive enhancement later

Volumetric temperature rendering at 25k–500k cells is comfortably within WebGL2 raymarching capability, and WebGL2 works on effectively every device a Philippine contractor’s office owns. WebGPU support is still uneven on the long tail of hardware. Three.js abstracts the difference: written correctly, the R3F viewer can adopt the WebGPURenderer later with minimal component changes. Ship compatibility first, adopt the faster renderer when it’s free to do so.

D5 — No new queue infrastructure

Cloud Run Jobs is the compute queue; the runJobs subcollection in Firestore (already built, with status, residuals, snapshots, and rules) is the job state. BullMQ+Redis would duplicate both while adding a paid dependency and another security surface. The per-user rate limiting v3.0 wanted from BullMQ is already implemented at the API layer (evaluateRateLimit) and can be extended per-route without any new infrastructure.

D6 — Languages

Language	Where	Role
TypeScript	Next.js app, engines, exporter, stores, 3D components	Everything user-facing and all business logic. The primary language of the platform.
Python 3.12	services/calc-engine (psychrometrics, ASHRAE endpoints) and services/cfd-solver (OpenFOAM wrapper)	Numerical/scientific glue where the ecosystem (fluidfoam, NumPy) is strongest.
GLSL	Raymarch volume shader, particle shaders inside R3F components	The only genuinely new language in v2.0 — small, contained, and unavoidable for volumetric rendering.
Bash / Docker / YAML	cfd-solver container, CI, deploy scripts	Infrastructure plumbing.

Explicitly not added: Rust (D3), Redis/Lua (D5), and no framework migration — Next.js + React + Zustand remain.

3. Target Architecture

One geometry pipeline, two solver tiers, one results contract, one viewer.

Stage	Component	State
Geometry	Floor/room polygons (Fabric-drawn or DXF-imported later) → building-geometry.ts → BuildingGeometryInput; single-room path → buildStructuredGrid voxel mesh	EXISTS
Case setup	SimulationCase (physics, solver profile, mesh snapshot) in Firestore with ownership rules	EXISTS
Tier A solve	Web-worker SIMPLE/k-ε with humidity; residuals + field snapshots streamed to Firestore	EXISTS
Tier B solve	buildOpenFOAMConfig → generateCaseFiles → GCS → Cloud Run Job (blockMesh → topoSet → createPatch → solver) → fluidfoam extraction → callback → importFieldData → same snapshot store	SCAFFOLDED — Section 4 completes it
Results	ArtifactManifest + RunFieldSnapshot (nx×ny×nz scalar/vector fields), solver-agnostic	EXISTS
Presentation	R3F viewer with volume/vector/particle/hotspot layers reading the same snapshots regardless of tier	Section 5 builds it

4. CFD Engine — From Scaffold to Engineering-Grade

The Wave 6/7 work established the skeleton. This section is the full build-out, restated as one coherent program with its verification gates. Nothing here is speculative infrastructure: every named store function, type, and file already exists in the repository unless marked NEW.

4.1 Phase C1 — Prove the solver locally (gate: one real room solves end-to-end)

- Build the cfd-solver image locally. The Dockerfile’s install method (dl.openfoam.com apt repo, openfoam2406-default, /usr/lib/openfoam/openfoam2406) was verified against openfoam.com’s current documentation in July 2026.
- Export one small real room (one inlet, one outlet, no obstructions) via a temporary debug script calling buildOpenFOAMConfig + generateCaseFiles, and run the container with plain volume mounts — no GCS, no callback.
- Expected first-run failure modes, in order of likelihood: degenerate boundary faces on unusual aspect ratios (read checkMesh output — the wrapper runs it non-fatally for exactly this reason); empty topoSet faceSets if a patch’s cells don’t touch a true outer face; divergence before the first write interval (relax fvSolution under-relaxation factors); fluidfoam shape mismatch (add an assertion comparing readmesh(structured=True) shape to nx/ny/nz before trusting any field).
- Gate to pass: output JSON contains non-NaN, physically plausible temperature/velocity/pressure arrays for that room.

4.2 Phase C2 — Cloud infrastructure

- Artifact Registry repo; push the proven image.
- GCS exchange bucket with a lifecycle rule deleting objects after ~7 days — job inputs/outputs are transient.

- Service account: roles/run.developer for the Next.js backend identity; roles/storage.objectAdmin scoped to that bucket only.
- gcloud run jobs create cfd-solver --cpu 4 --memory 8Gi --task-timeout 3600 --max-retries 0; tune after observing Phase C1's real memory usage.

4.3 Phase C3 — Orchestration (the two NEW routes)

Both routes were deliberately spec'd rather than written blind in Wave 6, because they execute authenticated cloud calls that couldn't be tested from the environment they were authored in. They are small once C1–C2 exist:

Route	Responsibility
POST /api/projects/[id]/simulations/[simId]/runs (NEW — GET exists)	Auth + ownership check (reuse the GET route's guard); ensure case.mesh (buildStructuredGrid if absent); buildOpenFOAMConfig → generateCaseFiles; createRunJob(source: 'openfoam'); upload {solver, files, dimensions, runJobId, callbackUrl, callbackSecret} JSON to GCS; trigger the Jobs run API with env overrides pointing CASE_INPUT_PATH / RESULT_OUTPUT_PATH at that object (GCS FUSE volume is the simplest wiring); return 202 with the run id. Never block the request on the solve.
POST /api/.../runs/[runId]/openfoam-callback (NEW)	Verify X-Callback-Secret; on completed → importFieldData(caseId, runJobId, 'openfoam', dimensions, data) then saveArtifactManifest + saveRunFieldSnapshot + updateRunJobStatus('completed'); on failed → updateRunJobStatus('failed', {errorMessage}). Idempotent: a duplicate callback for an already-terminal run is a no-op.

- UI change that rides along: CaseStatusBadge and SimulationRunProgressCard branch on source === 'openfoam' — lifecycle is queued → meshing → solving → done, not an iteration counter.
- Add solverBackend selection to case setup UI: Preview (instant, includes humidity) vs Engineering (minutes, temperature/velocity/pressure, defensible). Copy the honest trade-off into the UI itself.

4.4 Phase C4 — Validation ladder (the actual engineering-grade gate)

"Runs without crashing" and "engineering-grade" are different claims. This ladder is the difference, and its log is what you show a client's engineer who asks how you know:

Step	Check	Pass criterion
V1 Sanity	Buoyancy direction, outlet T between inlet T and wall T, no NaN/Inf anywhere	All hold on the C1 room
V2 Mass balance	Σ inlet volumetric flow vs Σ outlet flow at convergence	Imbalance < 2%
V3 Cross-solver	Same room through Preview and Engineering tiers; compare mean T and mean U	Same ballpark ($\pm 15\%$ mean T in °C-above-inlet terms) — not identical, different closures
V4 Hand calculation	Steady-state energy balance (inlet enthalpy + heat load) vs OpenFOAM mean room T	Within a few percent
V5 Breadth	Repeat V1–V4 on 3–4 different real rooms	All pass; keep the log
V6 Turbulence upgrade	Switch the case's turbulence model to k-omega-sst (exporter already maps it); re-run V1–V5	Passes — this is the moment v3.0's SST k- ω goal is achieved, via a dropdown

4.5 Phase C5 — Real geometry (snappyHexMesh)

Only after C1–C4: obstruction-aware meshing for furniture, racks, and interior walls. The voxel zones array in geometry-builder.ts already knows which cells are solid; the new work is a voxel→STL export (box-per-solid-cell is sufficient at these resolutions), a snappyHexMeshDict generator targeting that STL, and a branch in run_solve.py's mesh pipeline when solid zones are present. Treat as its own effort with its own validation pass — a meaningfully different meshing path deserves its own gate.

4.6 Explicit non-goals for the CFD engine in v2.0

- No humidity field on the OpenFOAM path (would require adding a scalar transport equation to the case setup; the Preview tier covers psychrometric questions until then). The result importer's humidity field stays optional for exactly this reason.
- No transient (time-accurate) simulation — steady RANS answers the design questions a BOQ needs.
- No conjugate heat transfer through walls — wall boundary conditions with fixed T / heat flux are the correct fidelity for room-level design.

5. 3D Visualization — React Three Fiber Overhaul

The current viewer (raw Three.js in BuildingSimulationViewer3D + CFDOverlay3D, ~1,150 lines) works and renders real geometry — the v3.0 audit line "no real geometry" is no longer true and was re-verified during Wave 6. What raw Three.js costs is composability: every layer (volume, vectors, particles, hotspots) is hand-managed imperative code. React Three Fiber makes each layer a component with props, which is what lets the viewer grow the v2.0 feature set without collapsing.

5.1 Migration strategy — strangler, not rewrite

- New R3F <SimulationCanvas> mounts alongside the old viewer behind a UI toggle. Old viewer is deleted only after the new one reaches feature parity (checklist: geometry, temperature overlay, velocity display, hotspots, camera controls, mobile touch).
- Dependencies: @react-three/fiber, @react-three/drei (already on three ~r128-compatible APIs in the repo — pin R3F/drei versions to the three version npm resolves, and note OrbitControls comes from drei, sidestepping the r128 OrbitControls gap noted in the codebase).
- Component tree: <SimulationCanvas> → <BuildingShell> (extruded rooms, wall thickness by construction type) → <TemperatureVolume> → <VelocityGlyphs> → <SupplyAirParticles> → <HotspotMarkers> → <SectionPlane> → <ComfortOverlay>. Each takes the RunFieldSnapshot data as props and nothing else — solver-agnostic by construction.

5.2 Layer specifications

Layer	Technique	Notes
TemperatureVolume	Data3DTexture (Float or Half-Float, nx×ny×nz) + fullscreen-quad raymarch GLSL: fixed-step front-to-back accumulation, transfer function blue→jade→copper→red mapped to the field's min/max from the snapshot descriptor	WebGL2 texture size limits are generous vs 60 ³ -class grids; step count 64–128 is plenty. Opacity transfer function editable via a small UI ramp.
VelocityGlyphs	InstancedMesh cones; per-instance quaternion from velocity direction, length ∝ U with a clamped scale; density-stride slider (every Nth cell)	Instancing keeps 10k+ glyphs at one draw call.
SupplyAirParticles	GPU particle advection: positions in a ping-pong FBO, advected by sampling the velocity Data3DTexture in a	This replaces the current Math.random() seeding in

Layer	Technique	Notes
	simulation shader; seeded at inlet patch faces; colored by local T	CFDOverlay3D with physically meaningful seeding at diffusers — the single biggest visual-credibility upgrade.
HotspotMarkers	$T > \mu + 2\sigma$ cells clustered to centroids; emissive spheres, click → readout card (T, position, nearest room)	Hotspot data already computed by the building solver — presentation only.
SectionPlane	Clipping plane on volume + geometry, draggable gizmo (drei <TransformControls>)	The tool engineers actually use to inspect stratification.
ComfortOverlay	PMV/PPD computed per cell from T, U , humidity (Preview tier) or T, U + assumed RH (Engineering tier), rendered as a floor-plane heatmap at 1.1m seated height	Comfort, not raw temperature, is the language clients understand. Label assumed-RH clearly on the Engineering tier.

5.3 Performance budgets (hard limits, checked in review)

- 60 fps on a mid-range laptop iGPU for a 35k-cell room with volume + glyphs + 4k particles; 30 fps floor on a 2022-class Android tablet.
- Draw calls < 100 in the full-feature view; volume raymarch ≤ 128 steps; particle count adaptive (halve until frame budget met).
- LOD: > 30 m camera distance → room boxes only; 10–30 m → walls, no volume; < 10 m → everything. (Retains v3.0's LOD concept unchanged — it was correct.)
- All layers respect prefers-reduced-motion: particles pause, animations become fades.

6. UI & Animation System

v3.0's design tokens were good and are retained unchanged (jade accent, ink primary, copper for cost, destructive red, the light/dark pairs). v2.0 adds the motion system, the CFD-specific surfaces, and the rules that keep animation from becoming decoration.

6.1 Motion principles

- Library: Framer Motion (motion/react) for DOM; R3F's own frame loop for 3D. One motion vocabulary: durations 150ms (micro), 250ms (panel), 400ms (page); easing cubic-bezier(0.2, 0, 0, 1); nothing bounces in an engineering tool.
- Motion must encode meaning: state change, causality, or spatial continuity. If removing an animation loses no information, remove it.
- prefers-reduced-motion collapses everything to opacity fades. This is an accessibility requirement, not a preference.
- Numbers animate by counting (BOQ totals, TR values) only on first computation — re-renders show final values instantly. Copper color reserved for currency figures, as in v3.0.

6.2 The CFD tile & simulation surfaces

Surface	Specification
CFD tile (project dashboard)	Status chip driven by RunJob: Preview runs show live iteration count + residual sparkline (data already streams to Firestore); Engineering runs show queued →

Surface	Specification
	meshing → solving → done stepper. Completed tile shows a static thumbnail of the temperature field (render-to-texture capture on completion) — never an idle animated 3D canvas in a dashboard grid.
Run detail page	Left: 3D SimulationCanvas. Right rail: residual convergence chart (recharts, log-y), field min/max/mean table, solver metadata card (tier, turbulence model, mesh cell count, wall-clock). Tier badge always visible: PREVIEW in muted, ENGINEERING in jade.
Comparison view	Two snapshots side-by-side with linked cameras (same OrbitControls target broadcast). Primary use: before/after equipment placement, and Preview-vs-Engineering during validation (Section 4.4 V3 uses this view).
Failure states	A failed Engineering run shows the errorMessage and the logTail from the RunJob verbatim in a monospace disclosure — engineers debug from real logs, not from "Something went wrong."

6.3 Page-level polish pass (ordered by client visibility)

- Project detail: tab transitions (250ms slide+fade), BOQ Verified/Outdated/Integrity-failed badges (already built) get a one-time draw-in; export buttons' disabled state explains itself via tooltip (v3.0 spec, retained).
- Load calculation: component breakdown renders as a stacked horizontal bar (walls/roof/glass/solar/people/lights/equipment/ventilation) that assembles left-to-right on first calc — the single most persuasive visual in a client demo because it shows where the heat comes from.
- Equipment selection: recommendation cards stagger in (40ms offsets); price-overridden SKUs show a small copper dot with tooltip "Admin-adjusted price" — honesty in the UI about which prices were overridden.
- Admin portal: no decorative motion at all; only state-change feedback. Security surfaces should feel still.
- Skeletons for every async surface (drei <Html> fallbacks inside the canvas; standard skeletons outside). No spinners longer than 400ms without progress information.

7. Remaining Engine Work (non-CFD)

Item	Scope	Size
BOQ Section E — Diffusers & grilles	Quantity from duct layout output (airflow-duct-engine.ts already computes runs); type by space type; add to catalog constants + cost engine	Small
BOQ Section G — Controls	Thermostats per zone, CO ₂ sensors per ASHRAE 62.1 occupancy category, BMS points count; rules table + line items	Small-medium
Itemized fees (v3.0 I/J/K completion)	Labor by trade instead of flat 35%; engineering fee line; permit/compliance fees	Small — rates table + math
cost-engine material lookup bug pass	The pre-existing lookup issues flagged in Wave 5; do after the pricing fixes have soaked, with a regression smoke script like smoke-price-override.ts	Small but careful
RTS load method (v3.0 §6)	Full ASHRAE RTS with PH design DB (the v3.0 tables are a good spec). Honest sequencing: current CLTD/CLF is component-based and credible; RTS is an accuracy upgrade, not a correctness fix — schedule after CFD C4	Large

Item	Scope	Size
DXF import (v3.0 file I/O)	ezdxf in calc-engine: DXF → room polygons → existing geometry pipeline. IFC deferred (ifcopenshell is heavy; demand-driven)	Medium

8. Technology Stack After v2.0

Layer	Technology	Status
Framework / UI	Next.js (App Router) + React + Tailwind + shadcn/ui	KEEP (current)
State	Zustand stores (auth, project, simulation, workspace...)	KEEP
3D	Three.js + React Three Fiber + drei; GLSL shaders; WebGL2 (WebGPU later via renderer swap)	NEW: R3F/drei/shaders
Motion	Framer Motion (DOM) + R3F frame loop (canvas)	NEW
Charts	recharts (residuals, load breakdown)	KEEP
Forms/validation	zod (already used in validation/admin.ts) + react-hook-form where forms grow	KEEP/EXTEND
Load calc	TypeScript CLTD/CLF now; RTS per Section 7	KEEP → UPGRADE
CFD Preview	TypeScript SIMPLE/k-ε web worker (humidity included)	KEEP
CFD Engineering	OpenFOAM 2406 (buoyantSimpleFoam → kOmegaSST) in Cloud Run Jobs; fluidfoam extraction; Firestore runJobs as job state	NEW (scaffolded)
Calc services	Python 3.12 FastAPI (psychrometrics/ASHRAE endpoints); ezdxf later	KEEP/EXTEND
Data	Firestore + security rules + composite indexes (incl. loginEvents index added in Wave 6)	KEEP
Auth	JWT cookie + Firebase Auth + custom claims RBAC; email+IP lockout; TOTP MFA when Firebase console is configured	KEEP
Storage/queue	GCS exchange bucket + Cloud Run Jobs (no Redis, no BullMQ)	NEW (infra only)
Exports	docx/ExcelJS/pdf generation with BOQ hash manifest (existing snapshot system)	KEEP
CI	GitHub Actions: tsc, eslint, smoke scripts (price-override + future ones), nightly build of cfd-solver image	EXTEND

9. Implementation Timeline — sized for one developer

Ordered by dependency and by demo value. Estimates assume focused solo work; they are ranges, not promises — and the first OpenFOAM run (C1) is explicitly a debugging phase whose duration nobody can honestly predict to the day. Phases are sequenced so the platform is demonstrably better after every single phase, never mid-rewrite.

Phase	Contents	Estimate	Demo-able outcome
P0 Land Wave 6	Apply patch, deploy Firestore index, review money-math diffs, run smoke scripts	0.5 day	Overrides reach quotes; IP lockout live
P1 CFD local proof (C1)	Docker build, one-room export, local solve, debug	2–7 days (honest range)	A real OpenFOAM result JSON from your own geometry

Phase	Contents	Estimate	Demo-able outcome
P2 Cloud path (C2+C3)	Registry, bucket, job, two routes, UI status branch, tier selector	4–7 days	Click "Run Engineering simulation" in the app; result appears
P3 Validation (C4)	V1–V6 ladder incl. kOmegaSST switch; keep the log	3–5 days	The claim "engineering-grade" with evidence behind it
P4 R3F viewer core	Canvas, BuildingShell, TemperatureVolume, VelocityGlyphs, parity checklist, old viewer removed	7–10 days	The new 3D view — volumetric temperature on real rooms
P5 Viewer advanced	Particles (inlet-seeded), hotspot cards, section plane, comfort overlay, comparison view	5–8 days	The demo that sells the product
P6 UI/motion pass	Motion system, CFD tile, load-breakdown bar, polish per Section 6.3	4–6 days	The whole app feels finished
P7 BOQ completion	Sections E, G, itemized fees, cost-engine bug pass + smoke script	3–5 days	Full 11-section BOQ
P8 snappyHexMesh (C5)	Voxel→STL, snappy dict, mesh-path branch, its own validation	5–10 days	Obstruction-aware CFD
P9 RTS + DXF	Per Section 7, in either order by client pull	2–3 weeks combined	Import a floor plan; RTS-grade loads

A personal note on sequencing

P0–P3 are deep-work phases that fit in whatever schedule you have around graduation and meet prep — each has clear stopping points. P4–P5 (the viewer) reward longer uninterrupted stretches; they are natural post-July-28 work. Nothing in this plan requires burning the candle from both ends in the same month you're trying to total 480.

10. Testing, Verification & Quality Gates

- Every phase ends with: `npx tsc --noEmit clean`, `eslint clean`, and its smoke script green. The Wave 6 pattern (`scripts/ts/smoke-price-override.ts` registered as an npm script) is the template — cheap, behavioral, honest.
- New smoke scripts to add as phases land: `smoke-openfoam-export` (case-file structural checks — the Wave 6 version of this proved its worth by catching a real patch-name mismatch before it ever reached a solver), `smoke-boq-sections` (11 sections present, totals reconcile), `smoke-validation-ladder` (replays V1–V4 checks against a stored reference result).
- Adopt `vitest` for pure-function unit tests (pricing resolution, BOQ math, geometry helpers) — the codebase currently has zero test framework; the smoke scripts filled that gap tactically, `vitest` fills it structurally.
- CI: on PR — `typecheck + lint + unit + smokes`; nightly — `cfv-solver image build` (catches upstream OpenFOAM repo changes early).
- The validation log from Section 4.4 is a permanent artifact. Version it in the repo (`docs/validation-log.md`). It is the difference between marketing language and an engineering claim.

11. Risk Register

Risk	Likelihood	Mitigation
First OpenFOAM run reveals exporter assumptions that smoke tests couldn't catch	High (expected)	C1 is explicitly a debugging phase with the four likeliest failure modes pre-analyzed; fixes land in the exporter with new structural checks
Cloud cost surprise on Engineering runs	Medium	max-retries 0, task-timeout 1h, bucket lifecycle deletion, per-user run rate limit at the API layer; monitor first month's billing before pricing the tier
R3F migration stalls at partial parity	Medium	Strangler pattern with an explicit parity checklist; old viewer is deleted only at 100%, so the product never regresses
Volume rendering too slow on low-end client hardware	Low-medium	Step-count and particle-count adapt to measured frame time; LOD floor is boxes+glyphs which run anywhere
Solo-developer bandwidth (graduation, meet, AVE work)	Certain	Every phase independently shippable; plan explicitly survives being paused between any two phases
Scraped price feeds (v3.0 §7.2) break or violate site terms	Medium	Deferred from v2.0 scope: manual admin overrides (shipped, audited) are the price-truth mechanism until a supplier relationship provides real feeds

12. Professional Assessment

The v3.0 document's core judgment — treat this as money-adjacent software — was right, and the platform now embodies it: audited prices, hash-locked BOQs, enforced ownership, real lockouts. Its weakness was that its most ambitious sections (custom SST $k-\omega$ solver, Rust geometry, WebGPU, Redis queues) specified impressive technology rather than a reachable path, and for a solo developer those two things are opposites.

v2.0's bet is different: spend your innovation budget where clients can see it (the viewer, the comfort overlay, the load-breakdown story) and where correctness is provable (the validation ladder), and rent everything else from tools with decades of validation behind them. OpenFOAM gives you v3.0's turbulence-model ambition as a configuration option. React Three Fiber gives you a viewer that can grow. The tier system turns the solver split into pricing structure instead of technical debt.

One caution worth keeping in writing: nothing in this plan is "engineering-grade" until Section 4.4's ladder passes and its log exists. Hold every future version of this document — and every demo you give — to that standard. It is the habit that will separate this product from every other estimation tool in the market it enters.

Prepared as the working successor to HVAC System Overhaul Plan v3.0. Grounded in commit 06b83d5 plus the Wave 6 patch (wave6-changes.patch).